

Trabalho Prático 1: Alinhamento Estratégico de Carteiras Recomendadas

Disciplina: Projeto e Análise de Algoritmos

Instituição: IFSULDEMINAS – Campus Poços de Caldas

Docente: Prof. Douglas Castilho

Estudante: Denysson Gomes da Silva

1. Descrição e Modelagem da Solução

1.1. Contexto do Problema

O objetivo é determinar o Núcleo de Consenso Ordenado entre duas carteiras de investimentos recomendadas (Corretora A com M ativos e Corretora B com N ativos). O consenso consiste na maior sequência de ativos compartilhada entre ambas, preservando rigorosamente a ordem de prioridade original sem inversões.

1.2. Modelagem Teórica: Maior Subsequência Comum (LCS)

O problema foi formalmente modelado como a Maior Subsequência Comum (Longest Common Subsequence - LCS).

Uma abordagem por força bruta ou recursão ingênua possui complexidade exponencial $O(2^{M+N})$, inviável para instâncias com $1 \leq M, N \leq 1000$. A solução foi implementada utilizando **Programação Dinâmica Bottom-Up (Tabulação)**, apoiada em duas propriedades:

- Subproblemas Sobrepostos:** Subsequências de prefixos menores são consultadas repetidamente. A matriz armazena os cálculos intermediários, evitando reprocessamento.
- Subestrutura Ótima:** O alinhamento ótimo dos prefixos de tamanho i e j depende unicamente dos resultados ótimos previamente calculados.

1.3. Equação de Recorrência

Sendo $L[i, j]$ o tamanho da LCS entre os prefixos das listas:

$$L[i, j] = \begin{cases} 0, & \text{se } i = 0 \text{ ou } j = 0 \\ L[i - 1, j - 1] + 1, & \text{se } i, j > 0 \text{ e } A[i - 1] = B[j - 1] \\ \max(L[i - 1, j], L[i, j - 1]), & \text{se } i, j > 0 \text{ e } A[i - 1] \neq B[j - 1] \end{cases}$$

1.4. Reconstrução da Sequência (Backtracking)

A dimensão do consenso (K) reside na célula $L[M, N]$. A recuperação dos ativos é feita rastreando de volta da posição (M, N) até a base:

- Se houver igualdade ($A[i - 1] == B[j - 1]$), o ativo faz parte do consenso, sendo registrado e o caminho recua na diagonal ($i \leftarrow i - 1, j \leftarrow j - 1$).
 - Se forem diferentes, o algoritmo segue em direção ao maior valor adjacente: para cima ($i \leftarrow i - 1$) se $L[i - 1, j] \geq L[i, j - 1]$, ou para a esquerda ($j \leftarrow j - 1$).
-

2. Análise de Complexidade Assintótica

2.1. Complexidade de Tempo

- **Construção e Preenchimento da Matriz:** A matriz $(M + 1) \times (N + 1)$ é percorrida por dois laços aninhados com operações elementares $O(1)$, totalizando custo de $\Theta(M \times N)$.
- **Reconstrução (Backtracking):** O laço de rastreamento decrementa ao menos um índice por iteração, consumindo no máximo $O(M + N)$.
- **Complexidade Total de Tempo:** $O(M \times N)$. Para $M, N = 1000$, a execução exige cerca de 10^6 operações, rodando em poucos milissegundos na JVM.

2.2. Complexidade de Espaço

- **Matriz de Tabulação:** Uma grade de inteiros de dimensões $(M + 1) \times (N + 1)$, exigindo espaço $O(M \times N)$.
 - **Vetores de Entrada e Resposta:** Ocupam $O(M + N + K)$.
 - **Complexidade Total de Espaço:** $O(M \times N)$. Para o limite superior de 1000 elementos por corretora, o consumo é de aproximadamente 4 MB, compatível com o heap padrão de memória.
-

3. Declaração do Uso de Inteligência Artificial

Conforme previsto na Seção 5 das instruções do trabalho prático:

- **Ferramentas utilizadas:** Google Gemini e GitHub Copilot.
- **Etapas de aplicação:**
 - Apoio no esclarecimento conceitual da modelagem por Maior Subsequência Comum (LCS).
 - Justificativa teórica da necessidade de complexidade polinomial $O(M \times N)$ frente ao limite de tempo em instâncias de até 1000 elementos.
 - Auxílio na depuração da rotina de recuperação por *backtracking* (ordenação dos índices do vetor de saída).
 - Formatação e estruturação da documentação técnica.